

Module 2 — Integration Task: PyTorch Housing

Price Prediction

Assignment name: M2-12-pytorch Repo: Your [M2-12-pytorch](#) assignment repo — branch [integration-2/pytorch](#) Submission: PR URL from [integration-2/pytorch](#) to [main](#) Grading: Autograded (CI check) + rubric-graded on 4 dimensions by TA

Objective

Apply PyTorch to a tabular regression problem. You will load a clean dataset, standardize features, convert data to tensors, define a neural network model, train it for 100 epochs, observe the loss decreasing, and save predictions to a CSV file.

This integration connects what you practiced in the lab (loading and processing data) with the deep learning constructs from the reading and concept session (tensors, `nn.Module`, training loop). By the end, you will have written and run your first real PyTorch training script.

Dataset

Your starter repo includes [data/housing.csv](#) — a synthetic dataset of Jordanian apartment properties.

Column	Type	Description
<code>area_sqm</code>	float	Apartment area in square meters (50–250)
<code>bedrooms</code>	int	Number of bedrooms (1–5)
<code>floor</code>	int	Floor number (1–15)
<code>age_years</code>	float	Building age in years (0–40)
<code>distance_to_center_km</code>	float	Distance to city center in km (0.5–25)
<code>price_jod</code>	float	Target variable — price in Jordanian Dinars (30K–150K)

The dataset is intentionally small (~200 rows) and clean (no missing values) so that training completes in seconds on CPU and you can focus on the PyTorch workflow rather than data cleaning.

Install PyTorch

PyTorch requires a special install command because it has CPU-only and GPU-specific builds. Run:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Verify the install:

```
import torch
print(torch.__version__)
print("CUDA available:", torch.cuda.is_available()) # Should print: False
```

Getting Started

Accept the assignment: [Accept M2-12-pytorch on GitHub Classroom](#) — this creates your personal starter repo with all files already in place.

In your assignment repo:

```
git checkout -b integration-2/pytorch
```

Your starter files are already in the repo:

```
train.py      # Script skeleton – fill this in
data/         # Your dataset
housing.csv   # Your dataset
requirements.txt # numpy, pandas, pytest (torch not listed – use the install)
```

What to Build:

train.py

Write a complete training script with the following structure. Your starter file has `TODO:` markers at each step.

1. Load Data

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn

# Load the CSV
df = pd.read_csv('data/housing.csv')
```

2. Separate Features and Target

```
# Features: all columns except price_jod
feature_cols = ['area_sqm', 'bedrooms', 'floor', 'age_years', 'distance_to_center_km']
X = df[feature_cols]
y = df[['price_jod']] # Keep as DataFrame with shape (N, 1)
```

3. Standardize Features

Subtract the mean and divide by the standard deviation for each feature column:

```
X_mean = X.mean()
X_std = X.std()
X_scaled = (X - X_mean) / X_std
```

Why this step matters: Features have very different scales — `area_sqm` ranges 50–250, `price_jod` ranges 30,000–150,000. Without standardization, gradient updates are dominated by large-scale features and training may be slow or unstable.

4. Convert to Tensors

```
X_tensor = torch.tensor(X_scaled.values, dtype=torch.float32)
y_tensor = torch.tensor(y.values, dtype=torch.float32)

print(f"X shape: {X_tensor.shape}") # Should be torch.Size([200, 5])
print(f"y shape: {y_tensor.shape}") # Should be torch.Size([200, 1])

dtype must be torch.float32 — PyTorch model weights are float32 by default. Using float64 (NumPy's default) causes a dtype mismatch error during training.
```

5. Define the Model

Define a class `HousingModel` that inherits from `nn.Module`:

```
class HousingModel(nn.Module):
    def __init__(self):
        super().__init__()
        # TODO: define three layers:
        # - nn.Linear(5, 32)  → 5 input features → 32 hidden units
        # - nn.ReLU()         → activation function
        # - nn.Linear(32, 1)  → 32 hidden units → 1 output (price prediction)

        self.forward = self._forward

    def _forward(self, X):
        # TODO: implement the forward pass
        # Pass x through layer1, then ReLU, then layer2
        # Return the output
        pass
```

The architecture: 5 inputs → 32 hidden units (with ReLU) → 1 output.

6. Instantiate Model, Loss, and Optimizer

```
model = HousingModel()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

7. Training Loop

Train for 100 epochs. Print the loss every 10 epochs:

```
num_epochs = 100

for epoch in range(num_epochs):
    # Forward pass: run inputs through the model
    predictions = model(X_tensor)

    # Compute loss: how wrong are the predictions?
    loss = criterion(predictions, y_tensor)

    # Reset gradients – MUST happen before backward
    optimizer.zero_grad()

    # Backward pass: compute gradients
    loss.backward()

    # Update weights
    optimizer.step()

    # Print progress every 10 epochs
    if epoch % 10 == 0:
        print(f"Epoch {epoch:3d}: Loss = {loss.item():.4f}")
```

You should see the loss decrease from a large initial value to a smaller final value as training progresses.

8. Save Predictions

After training, generate predictions on the full dataset and save them:

```
# Generate predictions (no gradient tracking needed for inference)
with torch.no_grad():
    predictions_tensor = model(X_tensor)

# Convert to numpy and create a DataFrame
predictions_np = predictions_tensor.numpy().flatten()
actuals_np = y_tensor.numpy().flatten()

results_df = pd.DataFrame({
    'actual': actuals_np,
    'predicted': predictions_np
})

results_df.to_csv('predictions.csv', index=False)
print("Saved predictions.csv")
```

9. Guard and Entry Point

```
if __name__ == "__main__":
    # ... all the code above goes inside main() ...
    main()
```

Structure all your code inside a `main()` function, and use the `if __name__ == "__main__":` guard.

Documentation Requirement

Create or update a `README.md` (or create `integration_notes.md`) that documents:

- What the model predicts** — describe the target variable and the 5 input features
- Training configuration** — number of epochs, learning rate, optimizer name, loss function
- Training outcome** — did the loss decrease? What was the final loss value?
- One behavioral observation** — describe something you noticed about the training process (examples: "Loss decreased rapidly in the first 20 epochs then leveled off", "Initial loss was very high because features were not standardized first", "Predictions underestimate prices above 120,000 JOD")

Committing and Submitting

When `train.py` runs end-to-end and produces `predictions.csv`:

```
git add train.py predictions.csv README.md # or integration_notes.md
git commit -m "Integration 2: PyTorch housing price prediction"
git push -u origin integration-2/pytorch
```

Open a PR from `integration-2/pytorch` to `main` in your integration assignment repo.

Your PR description must include:

- Model architecture (layers, activation, loss function, optimizer, learning rate)
- Training output — at minimum: loss at epoch 0, epoch 50, and epoch 100
- One-sentence observation about the training behavior
- Paste your PR URL into TalentLMS → Module 2 → Integration 2 to submit this assignment.

What the Autograder Checks

The CI workflow verifies:

Test	What it validates
<code>test_train_file_exists</code>	<code>train.py</code> is present
<code>test_train_runs_end_to_end</code>	<code>python train.py</code> exits with code 0
<code>test_training_output_shows_loss</code>	stdout contains "Epoch" and "Loss"
<code>test_loss_decreases</code>	The final reported loss is lower than the first reported loss
<code>test_predictions_file_exists</code>	<code>predictions.csv</code> is created after training
<code>test_predictions_has_correct_columns</code>	<code>predictions.csv</code> has <code>actual</code> and <code>predicted</code> columns

The autograder installs torch CPU-only automatically when it runs on your PR — you do not need to add torch to `requirements.txt`.

What TAs Are Looking For

Your submission is rubric-graded on four dimensions:

Dimension	What "Proficient" looks like
<code>train.py</code> correctness	Script runs end-to-end; <code>HousingModel</code> inherits <code>nn.Module</code> with <code>__init__</code> and <code>forward</code> ; training loop follows the four-step pattern; feature standardization implemented
Training evidence	Loss decreases over 100 epochs; printed every 10 epochs; final loss is lower than initial
<code>predictions.csv</code>	File exists with <code>actual</code> and <code>predicted</code> columns; correct number of rows
Documentation	PR describes model architecture; reports training outcome; README/notes covers all 4 required items

Common Mistakes

Mistake	Fix
Tensor dtype mismatch	Always use <code>dtype=torch.float32</code> when creating tensors from DataFrames
Target shape mismatch	Use <code>y = df[['price_jod']]</code> (double brackets) to keep shape (N, 1), or use <code>y_tensor.reshape(-1, 1)</code>
Forgetting <code>optimizer.zero_grad()</code>	Gradients accumulate across iterations — training diverges without this
Forgetting feature standardization	Features with different scales cause unstable or slow convergence
Using <code>loss</code> in print instead of <code>loss.item()</code>	<code>loss</code> prints the full tensor with <code>grad_fn: .item()</code> extracts the Python float
Not using <code>torch.no_grad()</code> for inference	Not required to work, but is the correct practice for generating predictions after training

Challenge Tiers

These optional challenges deepen the integration work. The base assignment above is your graded deliverable. Tiers build on each other in complexity. No additional starter files are needed; extend your existing code or create new scripts in the same repo.

Challenge 1 — Evaluation and Visualization

Add proper model evaluation to your training script:

- Train/test split** — split your data 80/20 before training. Train on 80%, evaluate on 20%. Do not use the test set during training.
 - Metrics** — compute and print MAE (Mean Absolute Error) and R^2 (coefficient of determination) on both train and test sets. Use NumPy — no need for scikit-learn.
 - Predictions plot** — scatter plot of actual vs. predicted prices for the test set, with a diagonal reference line (perfect predictions would fall on this line). Save as `predictions_plot.png`.
 - Loss curve** — plot training loss over all 100 epochs. Save as `loss_curve.png`.
 - Overfitting discussion** — add a section to your README: Is the model overfitting? How do train vs. test metrics compare? What would you try next?
- What to produce:**
- Updated `train.py` with train/test split and metrics
 - `predictions_plot.png` and `loss_curve.png`
 - Updated README with overfitting analysis

Challenge 2 — Manual Backpropagation

Implement the same housing model (5 → 32 → 1, ReLU, MSE) **without** `nn.Module`, `loss.backward()`, or `optimizer.step()`. Use raw tensors and manual gradient computation only.

What this means concretely:

- Initialize weight matrices and bias vectors as raw `torch.Tensor` objects (with `requires_grad=False` — you compute gradients yourself)
 - Implement the forward pass as matrix multiplications: $h = \text{ReLU}(X @ W_1 + b_1)$, $y_{\text{pred}} = h @ W_2 + b_2$
 - Compute MSE loss manually
 - Compute all partial derivatives by hand: dl/dw_2 , dl/db_2 , dl/dw_1 , dl/db_1 (chain rule through ReLU)
 - Update weights with $W := W - lr * dW$
- Then compare: does the manual implementation produce the same loss trajectory as your `nn.Module` version over 100 epochs? They should be close (not identical due to initialization differences).
- Why this is genuinely hard:** Partial derivatives of MSE through a ReLU layer with matrix weights is a real mathematical exercise. The ReLU derivative is piecewise (0 or 1), and getting the matrix dimensions right for backprop through two layers requires careful shape reasoning.
- What to produce:**
- `train_manual.py` — the manual implementation
 - A comparison of loss curves from both versions (plot or table)

Challenge 3 — Custom Training Framework

Build a reusable `Trainer` class that accepts:

- Any `nn.Module` model
 - Any loss function (`nn.MSELoss`, `nn.L1Loss`, etc.)
 - Any optimizer (`Adam`, `SGD`, etc.)
 - A config dict with: `learning_rate`, `early_stopping_patience`, `log_every_n_epochs`, `checkpoint_path`
- The Trainer should:**
- Run the training loop with the provided model, loss, and optimizer
 - Track train loss (and optionally validation loss) each epoch
 - Implement `early_stopping` — stop training if validation loss hasn't improved for `patience` consecutive epochs
 - Save the best model checkpoint (lowest validation loss) using `torch.save()`
 - Produce a training report: JSON file with config, final metrics, epoch-by-epoch loss history, and a loss curve PNG
- Demonstrate on two problems:**
- The housing dataset from the base assignment
 - A second dataset of your choice (could be synthetic — e.g., $y = \sin(x) + \text{noise}$)

Engineering pattern: This is the same abstraction that Hugging Face `Trainer` (Module 6) and PyTorch Lightning implement. Designing a clean, reusable training loop forces you to think about API boundaries, configuration vs. code, and separation of concerns.

What to produce:

- `trainer.py` with your `Trainer` class
- Training reports (JSON + PNG) for both problems
- A brief README section explaining your design decisions

Module 2 of 12 — Applied AI & ML Systems

